COMP5008 Final Assignment

Biogas Feedstock Collection Network: Data Structures and Algorithms for Sustainable Waste Management

Semester 2, 2026 @ EECMS, Curtin University

1. Introduction

A renewable energy company is investigating opportunities to convert organic waste and by-products from the red meat industry into renewable energy through anaerobic digestion and biogas production.

Potential feedstock sources, such as farms, are geographically distributed across Australia. Efficiently managing information about these farms and planning potential feedstock collection activities requires appropriate data structures and algorithms.

In this assignment, you will design and implement a backend **Biogas Feedstock Collection Network** that manages farms information, represents transportation connections, prioritises feedstock collection requests, and produces collection performance reports.

The system consists of four integrated modules:

Module	Name	Role in Biogas Feedstock Network
1	Graph-Based Farm Network	Models the transportation network between farms and the proposed biogas hub
2	Hash-Based Farm Lookup	Stores and retrieves farms information efficiently
3	Heap-Based Feedstock Collection Scheduling	Prioritises feedstock collection requests using feedstock availability and transportation efficiency
4	Sorting Collection Records	Sorts collection records for end-of-period reporting

The purpose of the assignment is not to assess your knowledge of biogas engineering, agricultural production, or geographic information systems. The real-world scenario provides the context in which you apply data structures and algorithms.

2. Learning Objectives

This assignment aims to improve students' understanding and application of fundamental data structures and algorithms in solving a real-world sustainability problem.

By completing this assignment, you should be able to:

- Implement and analyse an adjacency-list graph with BFS, DFS, and Dijkstra's shortest-path algorithm.
- Design and implement a hash table with collision handling suitable for at least 40 records.
- Implement a scheduler that combines feedstock availability and estimated transportation time.
- Implement Merge Sort and Quick Sort and benchmark their performance under different input conditions.
- Integrate multiple data structures and algorithms into a coherent, testable system.
- Analyse the time and space complexity of implemented algorithms.
- Justify appropriate data structure and algorithm choices for a real-world computational problem.

3. Assignment Scenario and Dataset

3.1 Real-World Context

The assignment is inspired by a real-world challenge involving the identification and collection of geographically distributed organic feedstock sources for potential use in a biogas system.

The following public farm-location website provides the geographic context for the assignment:

Farm Transparency Project — Map

The website provides publicly available geographic information about farm locations across Australia. You are not assessed on web scraping, geographic information systems, or data collection techniques.

3.2 Collecting Farm Data

You must use the farm data from the above public farm-location website. Requirements:

- **One region.** Choose a single state/territory, or one contiguous area within a state (e.g., the South West of WA), and collect the farms data from that region. Your choice of region does not affect the marking criteria.

- **Submit the data.** Your CSV files are part of your submission (Section 9.2). Markers may check a sample of records against the website; datasets that are fabricated or do not meet these requirements will affect the marks for the modules that depend on them.

The provided `farm_dataset_sample.csv` is an example for your reference. At minimum, each farm record contains:

Field	Description
`ID`	Unique identifier for the farm
`Name`	Name or label of the farm/location
`Species`	Animal species kept
`Lat`	Latitude of the site
`Lng`	Longitude of the site
`Street Num`	Street number
`Street`	Road name
`Suburb`	Locality / town
`State`	Australian state or territory abbreviation
`Postcode`	Postcode
`Country`	Country

Important: Do not modify the `FarmID` values provided by the website.

3.3 Biogas Hub Location

The provided dataset does **not** include a biogas processing hub. You must decide the location of the proposed hub yourself (for example, a town, port or industrial area within or near the region covered by the dataset). Justify your choice of hub location in the technical report. The chosen location does not affect the marking criteria.

3.4 Transportation Network

The provided dataset does **not** include transportation connections or distances between farms, or between farms and the hub. You must develop your own transportation network:

- decide which farm locations are connected to each other and to the hub;
- assign a positive estimated transportation time in minutes to each connection; and
- ensure that the network satisfies the test-graph specification in Section 4.2.5.

You may base the connections and transportation times on the latitude and longitude of the farms, on road distances, or on another documented estimation method of your choice. Describe and justify how your network was constructed in the technical report.

3.5 Educational Modelling Data

Some attributes required for the computational tasks are not available from the source website. The assignment therefore uses **educational modelling attributes**:

- `EstimatedFeedstock` — supplied with each collection request (Module 3); you design these values yourself;
- `TransportTime` — derived from the transportation network you construct (Section 3.4);
- `CollectionPriority` — calculated by your program (Section 6.2).

These values create a computational model for this assignment only. They **must not be presented as actual measurements or production figures reported by the Farm Transparency Project**.

4. Module 1: Graph-Based Farm Network

4.1 Problem Description

The proposed biogas system needs to coordinate the transportation of potential feedstock from geographically distributed farm locations to a proposed processing hub.

The transportation network is represented as a **weighted, undirected graph**.

- **Vertices/nodes** represent farm locations and the proposed biogas processing hub.
- **Edges** represent transportation connections between locations.
- **Edge weights** represent estimated transportation time in minutes.

The graph should allow the system to determine whether locations are connected and to calculate the shortest transportation route between locations.

4.2 Tasks

4.2.1 Graph Construction

Implement a `Graph` class using an **adjacency list**.

You must:

- dynamically insert vertices;
- dynamically insert weighted undirected edges;
- ensure that an undirected edge is represented in both directions;
- The implementation should correctly handle:
    - duplicate locations;
    - duplicate edges;
    - isolated locations;
    - invalid locations.

Do not use a built-in graph library.

4.2.2 Breadth-First Search (BFS)

Implement **Breadth-First Search**.

Input: a source location (e.g., Biogas Hub).

Output: all reachable locations grouped by level.

4.2.3 Depth-First Search (DFS)

Implement **Depth-First Search**.

Input: source location.

Use DFS to determine whether the network contains a cycle.

Output:

- whether a cycle exists;
- if a cycle is detected, identify the locations involved in the detected cycle.

The implementation must correctly handle disconnected and isolated locations.

4.2.4 Dijkstra's Shortest-Path Algorithm

Input: source location and destination location.

Use Dijkstra's algorithm [1] to determine:

- the shortest transportation time;
- the corresponding path between the source and destination.

You must not use a built-in shortest-path or graph algorithm.

4.2.5 Test Graph Specification

Your test graph must contain at least:

- **10 vertices**;
- **12 weighted edges**;
- at least one cycle;
- at least one disconnected or isolated farm/location; and
- positive transportation-time weights.

Your graph should be sufficiently complex to demonstrate the behaviour of BFS, DFS, and Dijkstra's algorithm.

4.2.6 Expected Outputs

Your program should produce:

- a textual/visual graph structure;
- BFS output grouped by level;
- DFS cycle-detection output;
- the detected cycle/location information where applicable;
- Dijkstra shortest-path output;
- shortest transportation time; and
- appropriate messages for invalid or unreachable locations.

5. Module 2: Hash-Based Farm Lookup

5.1 Problem Description

The collection system needs to retrieve farm information efficiently. For example, when a collection request is received for `FarmID = f458e`, the system should be able to retrieve the corresponding farm record without scanning the entire dataset. A hash table will therefore be used for efficient farm lookup.

5.2 Tasks

5.2.1 Hash Table Design and Implementation

Implement a hash table using **one** of the following collision-handling strategies:

1. Chaining, using your own linked list implementation; or
2. Open addressing with linear probing/quadratic probing/double hashing.

You must justify your choice in the technical report.

Implement a modulo-based hash function.

Your report must explain:

- the hash function;
- table-size selection;
- prime-number considerations where applicable;
- collision behaviour; and
- expected and worst-case performance.

Do not use a built-in dictionary/map for the core hash-table implementation.

5.2.2 Farm Record

Each record should contain the fields mentioned in Section 3.2.

You are welcome to include additional fields where appropriate, based on your own design.

5.2.3 Core Operations (with excepted/average O(1) time complexity)

Implement:

`insert(record)`

Insert a farm record into the hash table.

Handle duplicate `FarmID` values appropriately.

`search(FarmID)`

Return the complete farm record when the ID exists.

If the ID does not exist, display an appropriate not-found message.

`delete(FarmID)`

Remove a farm record.

The program must correctly handle attempts to delete a non-existent record.

5.2.4 Collision Handling

Demonstrate at least one collision.

Your output should clearly show how the collision is resolved.

- For chaining, show the relevant chain.
- For probing, show the probe sequence.

Calculate and display the load factor after every 10 insertions.

Discuss and implement your resizing strategy when load factor exceeds a threshold (e.g., 0.7).

5.2.5 Validation

Your implementation should validate:

- unique `FarmID`;
- valid IDs;
- non-empty farm names;
- valid numerical latitude and longitude values; and
- missing records.

Invalid input should not cause the program to terminate unexpectedly.

5.2.6 Testing Requirements

You must demonstrate:

- at least **40 farm records**;
- successful searches;
- unsuccessful searches;
- successful deletions;
- unsuccessful deletion attempts;
- at least one collision;
- load-factor changes; and
- the effect of deletion on subsequent searches.

5.2.7 Expected Output

- Log of inserts, searches, and deletes (concise, readable format).
- Explicit collision example(s) with intermediate states (e.g., probe sequences or chain contents).
- Summary of load factor behaviour and performance analysis.

5.2.8 Deliverables for Module 2

- Source code for hash table (with comments and clear function headers).
- Test driver demonstrating inserts, searches, deletes, and collision handling.
- Short written justification of hash strategy, table size, and hash function.

6. Module 3: Heap-Based Feedstock Collection Scheduling

6.1 Problem Description

A biogas collection system may receive multiple collection requests at the same time. The system therefore needs to determine which collection request should be processed first.

A collection request should consider both:

- the amount of potential feedstock available; and
- the estimated transportation time required to collect it.

The information required for this calculation should be obtained from the previous modules.

- Farm information is retrieved using the **Hash Table**.
- Transportation time is determined using the **Graph and Dijkstra's algorithm**.

Each collection request specifies a FarmID and the EstimatedFeedstock available for collection. EstimatedFeedstock is not part of the provided dataset: you must design the collection-request test data yourself, using reasonable educational modelling values (Section 3.5).

An array-based binary heap will then be used to maintain the collection requests according to their priority.

6.2 Priority Calculation

For each collection request, define:

- F = `EstimatedFeedstock` supplied in the collection request;
- T = shortest estimated transportation time in minutes.

Higher feedstock availability should increase priority, while longer transportation time should reduce priority. Design your own priority formula, where higher priority values indicate that a collection request should be considered earlier.

The exact priority values must be calculated by your program rather than manually entered.

6.3 Tasks

6.3.1 Retrieve Farm Information

For each collection request, you may:

1. retrieve the farm record using the hash table;
2. obtain the farm's location;
3. obtain the `EstimatedFeedstock` value from the collection request;
4. use the graph to determine the shortest transportation route to the processing hub; and
5. calculate the collection priority.

This is provided as an example only, and you are welcome to develop your own design.

6.3.2 Heap Implementation

Implement an **array-based binary heap**.

You may implement either:

- a Max Heap; or

- a Min Heap with appropriately transformed priority values.

Justify the choice of max vs min heap in the report.

Implement:

`insert(request)`

Add a collection request and restore the heap property using percolation.

Print the heap array after every insertion.

`peek()`

Return the highest-priority collection request without removing it.

`extract_priority()`

Remove and return the highest-priority collection request.

Restore the heap property using percolation.

Print the heap array after every extraction.

6.3.3 Edge Cases

Your system must handle:

- a farm with no path to the processing hub;
- a transportation time of zero;
- an invalid `FarmID`;
- missing farm records;
- invalid feedstock values;
- an empty heap;
- duplicate collection requests; and
- a request whose farm information has been deleted from the hash table.

For an unreachable farm, the program should clearly indicate that a valid collection route cannot currently be calculated.

6.3.4 Test Run

Demonstrate:

- at least **10 heap insertions**;
- at least **5 extractions**;
- heap state after every insertion;
- heap state after every extraction;
- calculated priority values; and
- the resulting collection order.

6.3.5 Expected Output

- Sequence of heap states after operations (readable array/tree printout).
- Log of priority computations.

6.3.6 Deliverables for Module 3

Submit:

- heap source code;
- collection-request test data;
- test driver;
- heap-state traces;
- priority calculations; and
- written explanation of the scheduling strategy.

7. Module 4: Sorting Collection Records

7.1 Problem Description

To maintain a complete record of collection requests, the system must store all collection requests in a database in a sorted order. These records can then be used for post-analysis and decision-making. You will implement and compare two sorting algorithms: Merge Sort and Quick Sort.

7.2 Tasks

7.2.1 Merge Sort

Implement Merge Sort. You may use either top-down Merge Sort or bottom-up Merge Sort.

Explain and justify your choice in the technical report.

7.2.2 Quick Sort

Implement Quick Sort. You must choose and justify a pivot-selection strategy.

Discuss how your pivot strategy can affect performance in the technical report.

7.2.3 Sorting Key

Sort collection records according to: **Estimated Transportation Time (T)** in **ascending order**.

7.2.4 Dataset Sizes and Input Conditions

You must test both sorting algorithms using: 100 records; 500 records; and 1,000 records.

For each dataset size, test the following three conditions: random, nearly sorted (≤10% of records displaced), and reversed.

Use a **fixed random seed** so that your experiments are reproducible.

7.2.5 Integrated Dataset Generation

The sorting experiment should be connected to the previous modules.

Where appropriate, the `Estimated Transportation Time` used in the collection records should be derived from the shortest-path information calculated using Module 1.

You should not simply create unrelated random numbers without explaining how the sorting dataset relates to the system.

7.2.6 Performance Analysis

For each dataset size and input condition, record the execution time for:

- Merge Sort;
- Quick Sort.

Present your results in a table. You may use an appropriate timing facility such as Python's `timeit` module for performance measurement.

You must **not** use a built-in sorting function to perform the sorting.

7.2.7 Analysis Questions

Your report should discuss:

1. How did Merge Sort and Quick Sort perform as the dataset size increased?
2. How did the input condition affect each algorithm?
3. How did your Quick Sort pivot strategy influence performance?
4. What are the theoretical time complexities of the two algorithms?
5. What are their worst-case behaviours?
6. What are the relevant space-complexity differences?
7. Which characteristics of the input data affect the observed performance?

Your discussion should distinguish between:

- theoretical time complexity; and
- experimentally observed execution time.

7.2.8 Expected Outputs

Provide:

- correctly sorted records;
- timing results;
- a comparison of Merge Sort and Quick Sort;
- discussion of performance across the three input conditions; and
- a brief reflection on algorithm choice.

7.2.9 Deliverables for Module 4

- Source code for merge sort and quick sort.
- Benchmark script and results (tables and optional plots).
- Technical report on stability and pivot strategy.

8. Integration of the Four Modules

The final system should demonstrate that the four modules are not completely independent programs. A UML diagram is required to illustrate the modules' interaction and data flow.

9. Submission Guidelines

9.1 Submission Deadline

30 October 23:59 (AWST). An additional 7 days for students with the Curtin Access Plan (CAP).

You must submit through **both required Blackboard submission links**. Submitting to only one link will be considered an incomplete submission.

Link	What to submit
Turnitin link	Technical report in PDF format only. The Declaration of Originality must be included as the first page of the report.
Assignment link	Single ZIP file named COMP5008_assignment_<studentID>.zip (see Section 9.2 for the required contents).

9.2 ZIP File Contents

Your ZIP file should **at least** contain:

```
COMP5008_assignment_<studentID>/
│
├── README.md
├── interactive_menu.py/.java
│
├── data/
│   ├── dataset.csv
│   └── ...
│
├── module1_graph/
│   ├── source code
│   └── test files
│
├── module2_hash/
│   ├── source code
│   └── test files
│
├── module3_heap/
│   ├── source code
│   └── test files
│
├── module4_sorting/
│   ├── source code
│   └── benchmark files
│
├── sample_output/
│   └── ...
│
└── technical_report.pdf
```

You are welcome to add additional files/folders based on your own design.

9.3 Source Code Requirements

You may use Python or Java. You may reuse appropriate code developed in practicals, provided that it is your own work and is appropriately self-cited in comments where required. For algorithmic implementations, cite appropriate references.

9.4 Prohibited Built-In Data Structures and Algorithms

You must implement the core data structures and algorithms yourself.

A 100% penalty applies to the affected module where a prohibited built-in is used to replace the required implementation.

Follow the additional programming restrictions provided in the unit materials.

9.5 Code Quality

All code must:

- be modular;
- use meaningful variable and function names;
- include appropriate comments;
- contain a header block identifying the author, date and purpose;
- include appropriate error/exception handling;
- validate important inputs; and

Criteria	Details	Marks
Insert, search and delete	All operations implemented correctly	3
Collision handling	Collision demonstrated with intermediate state	1
Hash function and scalability	Appropriate design; handles required dataset size effectively	2
Module 3 — Heap Scheduling		**6**
Insert and extract operations	Correct heap implementation; heap property maintained	2
Priority calculation	Correct integration of feedstock and transportation time	2
Heap-state tracing	Heap array printed after required operations	2
Module 4 — Sorting		**6**
Sorting correctness	Merge Sort and Quick Sort produce correctly sorted output	3
Performance analysis	Timing results across required sizes and input conditions; meaningful analysis	3
Technical Report		**8**
Overview and justification	Clear system purpose and justified data-structure choices	2
UML/data-flow diagram	Clear module interaction and data flow	2
Complexity analysis and reflection	Time/space analysis and performance trade-offs	2
Results, assumptions and limitations	Appropriate sample outputs, results, assumptions and limitations	2
Code Quality and Compliance		**6**
Modular, readable and documented code	Appropriate structure, naming, comments and compliance with restrictions	2
Testing and edge cases	Required test cases and relevant edge cases demonstrated	2
Error handling	Appropriate input validation and exception/error handling	2
TOTAL		**40**

13. Final Notes

You are encouraged to make reasonable design decisions where the specification permits flexibility, but you must clearly explain and justify those decisions in your technical report.

Good luck, and enjoy building your Biogas Feedstock Collection Network!

14. References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.